

核心工作流

使用 Batch API 批量处理任务

📄 复制页面



当你需要使用大语言模型处理大规模、低实时性要求的任务时，Batch API 是理想选择。它支持通过文件批量提交任务，相比实时 API 调用可以节省 40% 的推理费用。

⚠️ Batch API 支持 `kimi-k2.6` 和 `kimi-k2.5` 模型。这些模型的 `temperature`、`top_p` 等参数不可修改，请勿在请求 body 中设置这些参数。



创建批处理任务

上传 JSONL 文件并创建批处理任务



列出批处理任务

获取当前组织的批处理任务列表



获取任务详情

查询指定批处理任务的状态和详细信息



取消批处理任务

取消正在进行的批处理任务

使用流程

本指南通过一个文本分类的实例，展示 Batch API 的完整使用流程：

1. 构造输入文件

KIMI 开放平台 `"request-1", "method": "POST", "url": "/v1/chat/completions", "body":`

>

字段	是否必须	说明
<code>custom_id</code>	必须	自定义请求标识，用于追踪结果，需在文件内唯一
<code>method</code>	必须	请求方法，固定为 <code>POST</code>
<code>url</code>	必须	请求地址，固定为 <code>/v1/chat/completions</code>
<code>body</code>	必须	请求体，与 Chat Completions API 参数一致

⚠️ `body` 中的 `model` 必须是 `kimi-k2.6` 或 `kimi-k2.5`。这些模型的 `temperature`、`top_p`、`n`、`presence_penalty`、`frequency_penalty` 参数均不可修改，请勿在 `body` 中设置这些参数。

❗ 输入文件要求：

- 文件必须为 `.jsonl` 格式，大小不能为空且不超过 100MB
- 每行必须是合法的 JSON 对象，且包含 `custom_id`、`method`、`url`、`body` 四个字段
- `custom_id` 在文件内必须唯一
- 所有行的 `model` 必须相同，一个批次只允许一个模型
- `method` 固定为 `POST`，`url` 固定为 `/v1/chat/completions`
- 指定的模型必须存在且用户有访问权限

2. 上传文件

通过[文件上传接口](#)上传 JSONL 文件，`purpose` 必须设置为 `"batch"`。

KIMI 开放平台



```
import os
from openai import OpenAI
from openai.types import FileObject

>

client = OpenAI(
    api_key=os.environ.get("MOONSHOT_API_KEY"),
    base_url=os.environ.get("MOONSHOT_BASE_URL", "https://api.moonshot.cn/v1"),
)

file_object: FileObject = client.files.create(
    file=open("batch_requests.jsonl", "rb"),
    purpose="batch",
)

print(file_object.id) # 保存 file_id, 下一步使用
```

3. 创建任务

调用**创建批处理任务**接口，传入 `input_file_id` 和 `completion_window`。

`completion_window` 建议根据数据量合理设置，较长的时间窗口可以提高任务完成率。

KIMI 开放平台



```
import os
from openai import OpenAI
from openai.types import Batch
>

client = OpenAI(
    api_key=os.environ.get("MOONSHOT_API_KEY"),
    base_url=os.environ.get("MOONSHOT_BASE_URL", "https://api.moonshot.cn/v1"),
)

batch: Batch = client.batches.create(
    input_file_id="your_file_id",
    endpoint="/v1/chat/completions",
    completion_window="24h",
)

print(batch.id) # 保存 batch_id, 用于轮询状态
```

4. 等待完成

创建后任务进入 **validating** 状态，系统将异步校验输入文件。校验通过后进入 **in_progress** 状态开始执行。你可以通过[获取任务详情](#)接口轮询状态。

KIMI 开放平台



```
import os
import time

from openai import OpenAI
from openai.types import Batch

client = OpenAI(
    api_key=os.environ.get("MOONSHOT_API_KEY"),
    base_url=os.environ.get("MOONSHOT_BASE_URL", "https://api.moonshot.cn/v1"),
)

while True:
    batch: Batch = client.batches.retrieve("your_batch_id")
    completed: int = batch.request_counts.completed if batch.request_counts else 0
    total: int = batch.request_counts.total if batch.request_counts else 0
    print(f"状态: {batch.status} ({completed}/{total})")

    if batch.status == "completed":
        break
    elif batch.status in ("failed", "expired", "cancelled"):
        print(f"任务异常终止: {batch.status}")
        break

    time.sleep(10)
```

5. 处理结果

任务完成后，`output_file_id` 字段包含结果文件 ID，通过[获取文件内容](#)接口下载。如果有请求失败，`error_file_id` 包含错误文件 ID。

KIMI 开放平台



```
import json
import os

from openai import OpenAI

>

client = OpenAI(
    api_key=os.environ.get("MOONSHOT_API_KEY"),
    base_url=os.environ.get("MOONSHOT_BASE_URL", "https://api.moonshot.cn/v1"),
)

output = client.files.content("your_output_file_id")
for line in output.text.strip().split("\n"):
    result: dict = json.loads(line)
    custom_id: str = result["custom_id"]
    content: str = result["response"]["body"]["choices"][0]["message"]["content"]
    print(f"{custom_id}: {content}")
```

输出文件中每行对应一个请求的处理结果：

KIMI 开放平台

```
{
  "id": "request-1",
  "status_code": 200,
  "request_id": "",
  "body": {
    "id": "chatcmpl-xxx",
    "object": "chat.completion",
    "created": 1711475054,
    "model": "kimi-k2.6",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": "这段文本属于科技类。"
        },
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "prompt_tokens": 30,
      "completion_tokens": 10,
      "total_tokens": 40
    }
  },
  "error": null
}
```

完整代码示例

以下是将上述步骤串联起来的完整脚本，可直接复制运行：

 开放平台



```
import json
import os

import time

from pathlib import Path


from openai import OpenAI
```

... See all 78 lines

Batch 状态说明

状态

说明

validating

已创建，正在校验输入数据

failed

数据校验失败，任务终止

in_progress

数据校验通过，正在执行

finalizing

执行完毕，正在准备结果

completed

结果准备完毕，任务完成

expired

未在 completion_window 内完成

cancelling

已发起取消，等待实际取消

cancelled

取消完成，任务终止

任务管理

列出批处理任务

通过[列出批处理任务](#)接口查看当前组织下的所有批处理任务。

KIMI 开放平台



```
import os
from openai import OpenAI
from openai.pagination import SyncCursorPage
from openai.types import Batch

client = OpenAI(
    api_key=os.environ.get("MOONSHOT_API_KEY"),
    base_url=os.environ.get("MOONSHOT_BASE_URL", "https://api.moonshot.cn/v1"),
)

batches: SyncCursorPage[Batch] = client.batches.list(limit=10)
for batch in batches.data:
    print(f"{batch.id} - {batch.status} ({batch.request_counts.completed}/{batch.
```

取消批处理任务

通过[取消批处理任务](#)接口取消正在进行的任务。仅 `validating`、`in_progress`、`finalizing` 状态的任务可以取消。取消后任务状态会先变为 `cancelling`，最终变为 `cancelled`。

Python cURL Node.js



```
import os
from openai import OpenAI
from openai.types import Batch

client = OpenAI(
    api_key=os.environ.get("MOONSHOT_API_KEY"),
    base_url=os.environ.get("MOONSHOT_BASE_URL", "https://api.moonshot.cn/v1"),
)

batch: Batch = client.batches.cancel("your_batch_id")
print(f"状态: {batch.status}") # cancelling
```

KIMI 开放平台 Batch API 支持在输入文件中包含图片和视频内容。与文本任务的区别主要在于 **构造输入文件** 这一步，其余流程（上传、创建任务、轮询、处理结果）完全一致。

▸ [图片批处理示例](#)

▸ [视频批处理示例](#)

扩展建议

- 根据实际数据量调整 `completion_window`，较大的数据集建议设置 `3d` 或 `7d`
- 轮询间隔建议 10-60 秒，避免频繁请求
- 结果处理可以根据需求写入数据库或生成报告
- 建议对大文件做分批处理，每个文件控制在合理大小

此页面对您有帮助吗？

 是

 否

[◀ 文件问答指南](#)

[Tool Calls 说明 ▶](#)

